University Management System

Agile Software Engineering Project — Phase 1

Sprint Planning & Product Backlog

Team Members

Yehia Mohamed — 23P0067

Omar Tamer — 23P0096

Saeed Mohamed — 23P0255

Course: Agile Software Engineering

Date: April 2026

Contents

1. Introduction

This document is the Phase 1 submission for our University Management System (UMS) project. The idea behind the UMS is to bring the university's main processes—managing facilities, courses, staff, and communication—into one platform instead of having them scattered across different systems.

We are a team of three students following the Scrum framework. In this phase there is no coding involved. We focused on understanding the requirements, writing user stories, prioritizing the backlog, and planning our first sprint. Everything described here is also set up on our Jira board.

2. Scrum Team & Roles

Since we are only three members, everyone also contributes to development work. We assigned the main Scrum roles as follows for Sprint 1:

Name	Role (Sprint 1)	ID	Responsibilities
Yehia Mohamed	Product Owner	23P0067	Owns the backlog, sets priorities, clarifies backlog items and acceptance criteria, communicates with the TA/professor. Also writes code.
Omar Tamer	Scrum Master	23P0096	Facilitates Scrum events (planning, daily scrum, review, retro), ensures the time-box is respected, coaches the team on Scrum principles. Also writes code.
Saeed Mohamed	Developer	23P0255	Selects items for the sprint backlog, plans the work needed to create the increment, estimates effort and assesses capacity.

Note: As instructed by the course, we will rotate roles every sprint so that each member gets to experience being a PO, SM, and Developer. This is for learning purposes and does not happen in most real-world teams.

3. Prioritization Technique

3.1 Why MoSCoW?

We chose the MoSCoW method to prioritize our backlog. MoSCoW groups features into four buckets: Must Have (the system can't launch without these), Should Have (important but the system still works without it), Could Have (nice to have but won't hurt if left out), and Won't Have (out of scope for now).

We considered other approaches like the Impact vs. Effort matrix, but we picked MoSCoW for a few reasons:

- **It's simple:** All three of us could understand and apply it right away without needing training on scoring models.
- **It defines the MVP directly:** Must Have items are the MVP. This made our sprint planning much easier.

- **It fits Scrum well:** During sprint planning, we pull Must Have items first into the sprint, then Should Have if there's room.
- **Easy to communicate:** When presenting to the TA or professor, the categories make our priorities obvious.

3.2 How We Applied It

We went through every requirement in the project description together. For each one, we asked: "Without this feature, is the product viable?" If the answer was no, it's a Must Have. If it adds significant value but the system can still work without it, it's a Should Have. Nice-to-have features that can wait went to Could Have. We didn't mark anything as Won't Have for now since all features are eventually needed, but some (like payroll and performance tracking) are clearly for later sprints.

4. User Stories & Product Backlog

4.1 Writing Good User Stories

We followed the standard user story format taught in the course: "As a [type of user], I want [some goal], so that [some reason]." This ensures each story captures the who, what, and why.

We also checked our stories against the INVEST principles to make sure they're well-written:

- **Independent:** Each story is self-contained and can be developed without depending on other stories.
- **Negotiable:** The details of how to implement each story are open for discussion during sprint planning.
- **Valuable:** Every story delivers value to a specific user (admin, student, professor, etc.).
- **Estimable:** The team was able to estimate each story using story points.
- **Small:** Stories are small enough to be completed within a single sprint.
- **Testable:** Each story has clear acceptance criteria so we know when it's done.

Additionally, we applied the 3 C's concept from the course: each story has a Card (the story text in Jira), was discussed through Conversation (during our planning session), and has Confirmation (acceptance criteria that define when the story is considered complete).

4.2 MVP Definition

Our MVP (Minimum Viable Product) is the smallest set of features that makes the system usable for its main users (admins, professors, and students). It consists of the 10 Must Have items in our backlog, covering: user login with role-based access, classroom and lab record management with room booking, student records CRUD, course catalog with elective registration, material upload by professors, grade viewing for students, a staff directory, and a university announcements page. Together, these items touch all four modules at a basic level and form the working foundation that everything else will be built on top of.

4.3 Full Product Backlog

Below is our complete backlog. Story points were estimated using Planning Poker with the Fibonacci scale (1, 2, 3, 5, 8, 13). Each member picked a card privately, then we revealed simultaneously and discussed differences until we reached consensus.

ID	User Story	Priority	Module	Pts	SP	Status

US-01	As an admin, I want to add, edit, and delete classroom and lab records, so that the university's room data stays up to date.	Must Have	Facilities	5	1	To Do
US-02	As a professor, I want to view room availability and book a classroom for a date and time, so that I can reserve spaces for my classes.	Must Have	Facilities	5	1	To Do
US-03	As a staff member, I want to report a maintenance issue for a room, so that the facilities team can address it.	Should Have	Facilities	3	2	Backlog
US-04	As an admin, I want to manage student records (add, view, update, delete), so that student data is organized.	Must Have	Facilities	5	1	To Do
US-05	As an admin, I want to allocate equipment and software licenses to departments, so that resource usage is tracked.	Should Have	Facilities	3	2	Backlog
US-06	As an admin, I want to define and organize the course catalog (core and elective subjects), so that the academic structure is clear.	Must Have	Curriculum	5	1	To Do
US-07	As a student, I want to browse available courses and register for electives, so that I can plan my semester.	Must Have	Curriculum	5	1	To Do
US-08	As a professor, I want to upload assignments and course materials, so that students can access them online.	Must Have	Curriculum	5	1	To Do
US-09	As a professor, I want to create and grade assessments, so that I can evaluate student performance.	Should Have	Curriculum	5	2	Backlog
US-10	As a student, I want to view my grades and feedback for each course, so that I can track my progress.	Must Have	Curriculum	3	1	To Do
US-11	As an admin, I want to manage a directory of professors and TAs with their contact info and office hours, so that this information is centralized.	Must Have	Staff	5	1	To Do
US-12	As a TA, I want to view my assigned courses and responsibilities, so that I know what is expected of me.	Should Have	Staff	3	2	Backlog
US-13	As a manager, I want to track staff performance and publish faculty research, so that achievements are documented.	Could Have	Staff	5	2	Backlog

US-14	As a staff member, I want to view my payroll summary and submit leave requests, so that I can manage my HR needs.	Could Have	Staff	5	2	Backlog
US-15	As a parent, I want to view my child's progress and communicate with teachers through a portal, so that I stay informed.	Should Have	Community	5	2	Backlog
US-16	As a student, I want to message professors and schedule meetings, so that I can get academic guidance.	Should Have	Community	5	2	Backlog
US-17	As an admin, I want to post university-wide announcements and events, so that everyone stays informed.	Must Have	Community	3	1	To Do
US-18	As a user, I want to log in with my university credentials and see features based on my role, so that the system is secure.	Must Have	Cross-cutting	5	1	To Do

SP = Sprint number. Items marked Sprint 1 are pulled into the current sprint. Items marked Sprint 2 stay in the backlog for future sprints.

4.4 Acceptance Criteria (Sprint 1 Stories)

Each Sprint 1 story has acceptance criteria that act as the "Confirmation" in the 3 C's model. Basically, these are the checklist items that tell us when a story is actually done. If all criteria pass, we consider the story complete. Here they are:

ID	Story Summary	Acceptance Criteria
US-01	As an admin	AC1: Admin can add a new room with name, type, and capacity. AC2: Admin can edit an existing room's details. AC3: Admin can delete a room that is no longer in use. AC4: A confirmation message appears before deletion.
US-02	As a professor	AC1: Professor can see a list of available rooms filtered by date and time. AC2: Professor can book an available room. AC3: Double-booking the same room is prevented by the system. AC4: A confirmation is shown after a successful booking.
US-04	As an admin	AC1: Admin can add a new student with name, ID, program, and contact info. AC2: Admin can search and view a student's record. AC3: Admin can update student information. AC4: Admin can delete a student record with confirmation.
US-06	As an admin	AC1: Admin can add a course with name, code, type (core/elective), and credit hours.

		AC2: Admin can edit or remove courses from the catalog. AC3: Courses are displayed organized by type (core vs. elective).
US-07	As a student	AC1: Student can view the full list of available courses. AC2: Student can filter courses by type or department. AC3: Student can register for an elective course. AC4: System prevents registering for the same course twice.
US-08	As a professor	AC1: Professor can select a course and upload a file (PDF, DOCX, etc.). AC2: Uploaded materials appear under the correct course. AC3: Students enrolled in the course can view and download the materials.
US-10	As a student	AC1: Student can see a list of their enrolled courses with grades. AC2: Grades are displayed per assignment/assessment. AC3: If feedback was provided by the professor, it is visible to the student.
US-11	As an admin	AC1: Admin can add a staff member with name, role, email, office hours, and assigned courses. AC2: Admin can edit or remove staff entries. AC3: Anyone logged in can search and view the staff directory.
US-17	As an admin	AC1: Admin can create an announcement with a title, body, and date. AC2: All logged-in users can see announcements on the main page. AC3: Announcements are displayed in reverse chronological order.
US-18	As a user	AC1: User can log in with a valid university email and password. AC2: Invalid credentials show an error message. AC3: After login, the user sees only the features available to their role (admin, professor, student, etc.). AC4: User can log out and the session is ended.

4.5 Jira Board

All the backlog items above are created on our Jira board. The sprint is set up and the Must Have items are moved into Sprint 1.

Jira Project Link: https://yehyaelabyad.atlassian.net/jira/software/projects/UMS/boards/34

5. Splitting Large User Stories

5.1 Why Split?

Some of the requirements we got from the project description were just too big to fit into one user story. When a story is that large, it's hard to estimate, there's a good chance it won't get finished in one sprint, and it's tough to see how much progress you're actually making. So we had to break them down.

5.2 Techniques We Used

We used a few of the splitting techniques from the course lectures. Specifically:

- **Workflow Steps:** Breaking a large requirement down by the sequence of user actions involved.
- **Data Operations:** Splitting by Create, Read, Update, Delete (CRUD) operations.
- **Business Rules:** Separating stories based on different business logic or validation involved.

5.3 Example 1: Administrative Office Automation (Workflow Steps)

The original requirement said: "tools to support managing student records, generating transcripts, and handling admission applications." These are three clearly different workflows, so we split by Workflow Steps:

1. US-04: Manage student records (CRUD operations) — Sprint 1.
2. Generate academic transcripts — planned for a future sprint.
3. Handle admission applications — planned for a future sprint.

Each of these can be developed and tested independently, which is exactly what we want.

5.4 Example 2: Classroom & Lab Management (Data Operations + Business Rules)

The original requirement combined managing rooms, booking, viewing availability, and reporting maintenance all in one. We split it using Data Operations and Business Rules:

1. US-01: Manage room records (add, edit, delete) — basic CRUD operations.
2. US-02: View availability and book rooms — involves scheduling business rules and conflict checking.
3. US-03: Report maintenance issues — completely different data and workflow.

This way, US-01 and US-02 go into Sprint 1 as Must Haves, while US-03 (a Should Have) can wait until Sprint 2 without blocking anything.

6. Definition of Done (DoD)

The Definition of Done is basically our team's agreement on what "finished" actually means. It's different from acceptance criteria (which are specific to each story) — the DoD applies to every story in the sprint. We sat down and agreed that a story is "done" when:

1. The feature meets all the acceptance criteria defined for that story.
2. The code is reviewed, approved, and merged into the main branch in our Git repository.
3. Basic unit testing has been done and the feature has no obvious bugs.
4. Both functional and non-functional requirements are met (the feature works correctly and the UI is usable).
5. Any necessary documentation is updated (e.g., README, API notes).

6. The Product Owner has reviewed and accepted the feature.
7. The Jira ticket is moved to “Done” and the feature has been demonstrated during the sprint review.

We tried to keep the DoD realistic for a team of three students. We didn’t include things like automated CI/CD pipelines or deploying to a staging environment because honestly, that’s overkill for where we are right now. We can always tighten up the DoD in future sprints as we get more comfortable.

7. Sprint 1 Planning

Our sprint planning session was based on the steps from the Sprint Planning lecture. We time-boxed it to 4 hours, which is the max for a 2-week sprint according to the Scrum Guide.

7.1 Step 1: Set the Stage

Omar (Scrum Master) welcomed the team, stated the purpose of the meeting, and shared the agenda. Since this is our first sprint, there was no previous sprint to review or retrospective takeaways to discuss.

7.2 Step 2: Define the Sprint Goal

> ***Sprint Goal:*** *Build the foundation of the UMS by implementing user authentication with role-based access, basic room management and booking, student records, course catalog with registration, material upload, grade viewing, a staff directory, and an announcements page.*

This is basically the one sentence that sums up what we’re trying to achieve this sprint. It keeps us focused day to day. The way we implement it might change, but the goal itself stays the same.

7.3 Step 3: Present the Product Backlog

Yehia (Product Owner) presented the prioritized Product Backlog to the team. He explained each Must Have item and its acceptance criteria. Omar and Saeed asked clarifying questions about scope, requirements, and the expected value of each item.

7.4 Step 4: Break Down Stories into Tasks

The development team (all three of us, since PO and SM also code) analyzed the selected user stories and broke them into smaller technical tasks. For example, US-18 (Login) was broken into: design login page UI, set up authentication backend, implement role-based routing, and write basic tests.

7.5 Step 5: Estimate Effort

We used Planning Poker with the Fibonacci scale to estimate story points for each item. Each member selected a card privately, then we revealed simultaneously. When there were differences (for example, Omar gave US-02 an 8 while Saeed gave it a 5), we discussed the reasoning and re-voted until we reached consensus. Story points reflect complexity, effort, and uncertainty — not hours.

7.6 Step 6: Determine Team Capacity

We calculated our available time for the sprint. Each of us can realistically dedicate about 10–12 hours per week to this project (we have other courses and commitments). For a 2-week sprint with 3 members, that gives us roughly 60–72 total hours of available effort. We also accounted for some overhead from Scrum events (daily standups, review, retro) which reduces our focused development time slightly.

7.7 Step 7: Finalize the Sprint Backlog (The Commitment)

We looked at the total effort for the selected items and compared it against how much time we actually have. We felt pretty confident that the 10 Must Have stories (46 story points total) are doable within this sprint. All three of us agreed and committed to the Sprint Backlog.

7.8 Sprint Backlog

ID	User Story	Module	Pts	Assignee
US-01	As an admin, I want to add, edit, and delete classroom and lab records, so that the university's room data stays up to date.	Facilities	5	Yehia
US-02	As a professor, I want to view room availability and book a classroom for a date and time, so that I can reserve spaces for my classes.	Facilities	5	Yehia
US-04	As an admin, I want to manage student records (add, view, update, delete), so that student data is organized.	Facilities	5	Yehia
US-06	As an admin, I want to define and organize the course catalog (core and elective subjects), so that the academic structure is clear.	Curriculum	5	Omar
US-07	As a student, I want to browse available courses and register for electives, so that I can plan my semester.	Curriculum	5	Omar
US-08	As a professor, I want to upload assignments and course materials, so that students can access them online.	Curriculum	5	Omar
US-10	As a student, I want to view my grades and feedback for each course, so that I can track my progress.	Curriculum	3	Saeed
US-11	As an admin, I want to manage a directory of professors and TAs with their contact info and office hours, so that this information is centralized.	Staff	5	Saeed
US-17	As an admin, I want to post university-wide announcements and events, so that everyone stays informed.	Community	3	Saeed
US-18	As a user, I want to log in with my university credentials and see features based on my role, so that the system is secure.	Cross-cutting	5	All

- **Duration:** 2 weeks
- **Estimated velocity:** 46 story points
- **Items selected:** 10 user stories (all Must Have)

8. Anticipating Changes

One thing we kept in mind is that Agile is all about being okay with change. Our Product Backlog isn't set in stone — it's going to evolve as we learn more and get feedback. Here's how we tried to set ourselves up for that:

- **Independent stories:** Each user story is as self-contained as possible (following INVEST). Changing something in the Staff module shouldn't break a Facilities story.
- **Re-prioritization each sprint:** MoSCoW priorities are not permanent. A Should Have could become Must Have next sprint based on stakeholder feedback.
- **Open backlog:** New stories can be added anytime by the Product Owner. New requests go to the Product Backlog, not the current sprint (no scope creep).
- **Retrospective-driven improvements:** After Sprint 1, we will hold a retrospective to discuss what went well, what problems arose, and identify actionable improvements for the next sprint. This may lead to changes in estimation, work distribution, or even updates to our DoD.
- **Role rotation:** Changing roles each sprint means each member brings a fresh perspective, which naturally surfaces process improvements.

9. Jira Setup Summary

Our Jira project is configured as follows:

- **Project type:** Scrum board
- **Issue types:** Story (for user stories), Sub-task (for technical breakdowns like "design DB schema" or "build login page")
- **Board columns:** To Do → In Progress → In Review → Done
- **Labels:** Each story is labeled with its module name and MoSCoW priority
- **Story points:** Entered in the story point field for each issue
- **Acceptance criteria:** Written in the description of each Sprint 1 story
- **Sprint 1:** 10 stories moved in, sprint goal set, 2-week duration